

Olist 데이터셋 인사이트 도출 채점 기준

무엇을 보고 어떻게 점수를 매기는지 전부 공개합니다. 쓰기 전에 한 번, 내기 전에 한 번 읽어보세요.

이 과제는 쿼리의 화려함을 보지 않습니다. 테이블 2개만 조인해도 최고 단계가 나올 수 있고, 5개 테이블에 윈도우 함수를 써도 1단계가 될 수 있습니다. 보는 것은 하나입니다 — 세운 질문에 답하기 위해 무엇을 고민했는가.

0 제출물과 배점

네 가지를 제출합니다. 앞의 세 개가 서로 어긋나면 마지막이 아무리 좋아도 분석이 성립하지 않습니다.

- | | |
|---------|--|
| ① 질문 | 무엇을 알아보려 했는지 한 문장. <i>배점이 가장 큰 항목의 기준이 되므로 반드시 적습니다.</i> |
| ② 쿼리 | BigQuery에서 실제로 돌린 SQL |
| ③ 실행 결과 | 결과 표를 그대로 복사. 행이 많으면 상위 10~20행 |
| ④ 인사이트 | 무엇을 발견했고, 왜 그렇게 보며, 그래서 무엇을 하자는 것인지 |

30

근거 사슬
정합성

25

쿼리 설계
판단

20

분석의
깊이

15

비즈니스
해석

10

액션
제안

60점 이상이면 제출 권장 수준입니다. 점수는 성적이 아니라 “이 정도면 내도 되겠다”는 신호로만 쓰세요.

1 근거 사슬 정합성

30점

질문 → 쿼리 → 결과 → 주장이 한 줄로 이어지는지 봅니다. 네 고리 중 하나라도 끊기면 나머지가 아무리 좋아도 분석이 성립하지 않기 때문에 배점이 가장 큼니다.

단계	점수	기준
1	0-8	사슬이 끊김. 질문이 없거나, 쿼리가 질문과 다른 것을 계산하거나, 주장이 결과에 없는 내용임
2	9-16	질문과 쿼리가 대체로 맞지만 범위가 어긋남 (아래 표 참고)
3	17-24	질문·쿼리·결과·주장이 일치하고, 인용한 수치가 붙여넣은 결과에 실제로 있음
4	25-30	위에 더해 비교 기준선 (전체 평균·다른 세그먼트·이전 기간)을 제시해 “이 숫자가 큰지 작은지”에 답함

2단계로 내려가는 전형적인 어긋남

이렇게 물어놓고	이렇게 세면 다른 걸 센 것
“재구매 고객”	<code>customer_id</code> 로 셈. 이 컬럼은 주문마다 새로 생기므로 재구매가 잡히지 않습니다. 사람 단위는 <code>customer_unique_id</code>
“배송 지연”	<code>canceled</code> · <code>unavailable</code> 주문까지 포함. 배송된 적 없는 주문의 지연은 의미가 없습니다
“주문당 평균”	<code>order_items</code> 단위로 셈. 한 주문에 상품이 3개면 그 주문이 3번 계산됩니다
“카테고리별”	INNER JOIN 때문에 상품 정보가 없는 주문이 조용히 빠짐. 분모가 달라집니다
“전체 주문 중”	<code>order_items</code> 와 조인. 항목이 하나도 없는 주문 775건 이 사라집니다

2 퀴리 설계 판단

25점

기법의 개수가 아니라 **선택의 흔적**을 봅니다. JOIN을 더 붙인다고, 윈도우 함수를 쓴다고 점수가 오르지 않습니다.

단계	점수	기준
1	0-6	실행 불가하거나 스키마에 없는 컬럼 사용. 또는 질문에 답할 수 없는 구조
2	7-13	실행되고 답도 나오지만 기본값에 머물. 아래 판단 지점이 하나도 다뤄지지 않음
3	14-19	아래 판단 지점 중 둘 이상 을 의도적으로 처리함
4	20-25	위에 더해 왜 그렇게 선택했는지 인사이트나 주석에 밝힘. 다른 선택을 했다면 결과가 어떻게 달라지는지까지 쓰면 확실한 4단계

다섯 개의 판단 지점

지점	스스로에게 물어볼 것
모집단 정의	<code>order_status</code> 를 어떻게 다룰 것인가. <code>delivered</code> 만 볼 것인가, <code>canceled</code> 만 뺄 것인가. 내 질문이 묻는 “주문”은 어디까지인가
JOIN 종류	LEFT인가 INNER인가. 이건 문법 취향이 아니라 “ 한쪽에 없는 행을 남길 것인가 ”라는 질문입니다. 예: 리뷰를 안 쓴 주문도 분모에 넣을 것인가? 넣어야 하면 LEFT, 리뷰가 있는 주문만 보는 게 맞으면 INNER. 어느 쪽이든 의도한 것이어야 합니다
집계 단위	결과의 행 하나가 무엇인가 — 주문 1건? 주문상품 1건? 고객 1명? <code>COUNT(DISTINCT order_id)</code> 와 <code>COUNT(*)</code> 는 다른 질문에 답합니다
고객 식별자	<code>customer_id</code> (주문마다 새로 생김) 와 <code>customer_unique_id</code> (사람) 중 내 질문에 맞는 것은 무엇인가
NULL 처리	배송되지 않아 <code>order_delivered_customer_date</code> 가 비어 있는 주문을 어떻게 할 것인가. 평균에서 조용히 빠지게 둘 것인가, 명시적으로 뺄 것인가

4단계로 가는 가장 쉬운 길: 인사이트 본문에 한 줄만 더 쓰면 됩니다. — “취소·미배송 주문은 배송 기간을 켈 수 없어 delivered 만 남겼습니다. 이걸 포함하면 표본이 약 3% 늘지만 지연일 계산이 불가능해집니다.”

3 분석의 깊이

20점

단계	점수	기준
1	0-5	한 가지 차원의 순위 나열에서 끝남 (“SP주 주문이 가장 많다”)
2	6-11	두 변수의 관계를 보거나, 시간에 따른 변화를 봄
3	12-16	세그먼트를 나눠 비교하거나, 결과를 왜곡할 수 있는 요인을 고려함
4	17-20	위에 더해 반대 가설 을 세워 데이터로 검증하거나 배제함

반대 가설의 예: “지연된 주문의 리뷰가 낮은 게 아니라, 원래 리뷰가 낮은 카테고리가 배송이 오래 걸리는 것일 수도 있다” → 카테고리를 고정하고 다시 본다.

4 비즈니스 해석

15점

단계	점수	기준
1	0-4	현상 서술만 있음. “그래서 무엇인지”가 없음
2	5-9	왜 그런지 가설을 제시함
3	10-13	가설이 데이터 특성이나 이커머스 도메인 맥락과 연결됨
4	14-15	인과를 단정하지 않고, 이 해석이 틀릴 수 있는 조건 을 함께 밝힘

5 액션 제안

10점

단계	점수	기준
1	0-2	없거나 “개선이 필요하다” 수준
2	3-6	방향은 있으나 대상·기준이 불명확
3	7-10	대상·기준·확인 방법이 구체적

단계	예시
1	“배송을 개선하자”
2	“배송 지연이 심한 주(state)를 관리하자”

3

“실제 배송이 예상일보다 평균 6일 이상 늦는 4개 주에 대해 예상 배송일을 재산정하고, 3개월 뒤 해당 주의 `review_score` 평균이 오르는지 확인한다”

아래 수치는 기준 설명을 위해 지어낸 예시이며 실제 Olist 데이터의 값이 아닙니다. 형식과 사고 과정만 참고하세요.

약한 제출물 · 합계 35점

질문

배송이 늦으면 리뷰가 안 좋은가?

쿼리

```
SELECT
  CASE WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date
    THEN '지연' ELSE '정시' END AS delay,
  AVG(r.review_score) AS avg_score
FROM olist_raw.orders AS o
JOIN olist_raw.order_reviews AS r ON o.order_id = r.order_id
GROUP BY delay;
```

결과

지연 2.6 / 정시 4.3

인사이트

지연되면 평균 리뷰가 낮다. 배송을 개선해야 한다.

근거 사슬 **14**(2단계 · 취소·미배송 주문을 다루지 않았고 비교 기준선 없음) · 쿼리 설계 **10**(2단계 · 판단 지점 미처리) · 값이 **5**(1단계) · 해석 **4**(1단계) · 액션 **2**(1단계)

질문

배송이 예상일보다 늦게 도착한 주문은 정시 도착 주문보다 리뷰 점수가 낮은가? 낮다면 며칠부터 두드러지는가?

쿼리

```
WITH delivered AS (
    -- 배송 완료 건만: 취소·미배송은 지연일을 켤 수 없음
    SELECT
        order_id,
        DATE_DIFF(
            DATE(order_delivered_customer_date),
            DATE(order_estimated_delivery_date), DAY) AS delay_days
    FROM olist_raw.orders
    WHERE order_status = 'delivered'
    AND order_delivered_customer_date IS NOT NULL
)
SELECT
    CASE WHEN d.delay_days <= 0 THEN '정시 이내'
    WHEN d.delay_days <= 5 THEN '1-5일 지연'
    WHEN d.delay_days <= 10 THEN '6-10일 지연'
    ELSE '11일 이상 지연' END AS delay_bucket,
    COUNT(DISTINCT d.order_id) AS order_cnt,      -- 주문 단위 (아이템 아님)
    ROUND(AVG(r.review_score), 2) AS avg_score
FROM delivered AS d
JOIN olist_raw.order_reviews AS r ON d.order_id = r.order_id -- 리뷰 있는 주문만
GROUP BY delay_bucket
ORDER BY MIN(d.delay_days);
```

결과 (예시)

구간	주문 수	평균 리뷰
정시 이내	88,120	4.28
1-5일 지연	4,930	3.41
6-10일 지연	1,780	2.55
11일 이상 지연	1,640	1.92

인사이트

전체 평균 리뷰는 **4.09**인데, 1-5일 지연 구간은 3.41로 이미 평균을 밑돕니다. 다만 하락이 가파른 지점은 **6일부터**, 2.55까 지 떨어져 정시 대비 1.7점 낮습니다. “늦으면 리뷰가 낮다”가 아니라 “**6일을 넘기면 급락한다**”가 정확한 표현입니다. 약속 대비 편차가 임계치를 넘는 순간 고객이 이를 ‘사고’로 인식하는 것으로 보입니다.

설계 선택: 지연일을 켤 수 없는 취소·미배송 건은 제외했습니다. 리뷰를 쓰지 않은 주문도 INNER JOIN으로 뺐는데, 리뷰 점수가 종속변수이므로 없는 행은 분석 대상이 아니기 때문입니다. 다만 이 선택 때문에 **불만이 커서 아예 리뷰를 안 쓴 고객**이 빠졌을 수 있어, 실제 지연의 악영향은 위 수치보다 클 가능성이 있습니다.

반대 가설 검증: 배송이 오래 걸리는 카테고리가 원래 리뷰가 낮은 것일 수 있어 가구·대형가전을 제외하고 다시 집계했으나 구간별 격차는 유지되었습니다.

액션: 6일 이상 지연 비중이 높은 상위 4개 주에 대해 예상 배송일을 현재보다 보수적으로 재산정하고, 3개월 뒤 해당 주의 지연 구간 분포와 평균 리뷰가 개선되는지 확인합니다.

근거 사슬 **26**(4단계 · 전체 평균 기준선 제시) · 쿼리 설계 **22**(4단계 · 모집단·집계 단위·JOIN 종류를 처리하고 이유를 밝힘) · 깊이 **14**(3단계) · 해석 **11**(3단계) · 액션 **8**(3단계)

두 제출물의 쿼리 난이도 차이는 크지 않습니다. 강한 쪽도 테이블 2개를 조인했을 뿐입니다. 차이는 WHERE `order_status = 'delivered'` 한 줄, 구간을 나눈 판단, 그리고 왜 그렇게 했는지 적은 세 문장에서 나왔습니다.

7 제출 전 자가 점검

항목	스스로 확인
근거 사슬	내 쿼리를 말로 옮기면 무엇을 세는 문장이 되는가? 그게 내가 적은 질문과 같은가?
근거 사슬	인사이드에 쓴 숫자가 붙여넣은 결과 표에 실제로 있는가?
근거 사슬	이 숫자가 큰 건지 작은 건지 알려주는 비교 대상이 있는가?
쿼리 설계	<code>order_status</code> 를 한 번이라도 생각했는가?
쿼리 설계	LEFT / INNER 선택이 의도한 것인가, 그냥 쓴 것인가?
쿼리 설계	결과와 행 하나가 무엇 단위인지 한 마디로 말할 수 있는가?
쿼리 설계	그 선택의 이유를 인사이드에 한 줄이라도 적었는가?
깊이	순위 나열에서 끝나지 않았는가?
해석	“그래서 무엇”에 답했는가? 틀릴 수 있는 조건을 밝혔는가?
액션	대상·기준·확인 방법이 들어 있는가?

8 참고: 테이블과 데이터 함정

olist_raw 테이블

테이블	주요 컬럼
<code>orders</code>	<code>order_id</code> , <code>customer_id</code> , <code>order_status</code> , <code>order_purchase_timestamp</code> , <code>order_approved_at</code> , <code>order_delivered_carrier_date</code> , <code>order_delivered_customer_date</code> , <code>order_estimated_delivery_date</code>
<code>order_items</code>	<code>order_id</code> , <code>order_item_id</code> , <code>product_id</code> , <code>seller_id</code> , <code>shipping_limit_date</code> , <code>price</code> , <code>freight_value</code>
<code>order_payments</code>	<code>order_id</code> , <code>payment_sequential</code> , <code>payment_type</code> , <code>payment_installments</code> , <code>payment_value</code>
<code>order_reviews</code>	<code>review_id</code> , <code>order_id</code> , <code>review_score</code> , <code>review_comment_title</code> , <code>review_comment_message</code> , <code>review_creation_date</code> , <code>review_answer_timestamp</code>
<code>customers</code>	<code>customer_id</code> , <code>customer_unique_id</code> , <code>customer_zip_code_prefix</code> , <code>customer_city</code> , <code>customer_state</code>
<code>sellers</code>	<code>seller_id</code> , <code>seller_zip_code_prefix</code> , <code>seller_city</code> , <code>seller_state</code>

products	product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
geolocation	geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state
product_category_name_translation	product_category_name, product_category_name_english

미리 알려주는 함정

- **orders** 와 **customers** 는 **customer_id** 로 1:1 매칭됩니다. 재구매를 세려면 **customer_unique_id** 를 써야 합니다.
- **order_items** 에 행이 하나도 없는 주문이 **775건** 있습니다. 이 테이블과 조인하는 순간 조용히 사라집니다.
- 한 주문에 **order_item_id** 가 여러 개 붙을 수 있습니다. 주문 단위 집계와 상품 단위 집계를 섞으면 수치가 부풀려집니다.
- **order_status** 에는 **delivered** 외에 **canceled** , **unavailable** , **shipped** 등이 섞여 있습니다.
- 데이터 기간은 2016-09 ~ 2018-10 입니다. 양 끝 구간은 데이터가 얇으니 시계열을 볼 때 주의하세요.

셀프체크 도구 — 제출 전에 이 기준으로 자동 채점을 받아볼 수 있습니다. 항목별로 지금 몇 단계인지, 한 단계 올리려면 무엇을 해야 하는지 알려줍니다. 여러 번 고쳐서 다시 돌려도 됩니다. 링크와 접속 코드는 공지를 참고하세요.